

Deep Learning

Aplicações: Modelagem de Linguagem, Tradução Automática e Atenção

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

junho de 2026

Overview

1. Modelagem de Linguagem
2. LM com Redes Neurais
3. Decodificação
4. Avaliação
5. seq2seq
6. Atenção
7. Referências

Overview

1. Modelagem de Linguagem

2. LM com Redes Neurais

3. Decodificação

4. Avaliação

5. seq2seq

6. Atenção

7. Referências

O Que é Modelar?

- Modelar um fenômeno significa ter alguma capacidade preditiva sobre ele.
- Em *Language Modeling* queremos modelar a linguagem natural, ou seja, ser capazes de **computar a probabilidade de uma frase**.
- Em notação compacta, dada uma sequência de *tokens* $y_1, y_2, \dots, y_n$, queremos a probabilidade conjunta:

$$P(y_1, y_2, \dots, y_n)$$

Probabilidade de uma Frase

- Aplicando a regra da cadeia da probabilidade:

$$\begin{aligned} P(y_1, y_2, \dots, y_n) &= P(y_1) \cdot P(y_2 \mid y_1) \cdots P(y_n \mid y_1, \dots, y_{n-1}) \\ &= \prod_{t=1}^n P(y_t \mid y_{<t}) \end{aligned}$$

- Modelar a frase inteira se reduz a modelar, a cada passo, a distribuição do **próximo token** dado o **contexto** dos *tokens* anteriores.

Como Estimar Essas Probabilidades?

- Não temos meios práticos de computar $P(y_t \mid y_{<t})$ por contagem direta em corpus, sequências longas raramente se repetem.
- Diversas abordagens baseadas em *n-grams* foram usadas ao longo dos anos.
 - Aproximam $P(y_t \mid y_{<t})$ por $P(y_t \mid y_{t-n+1}, \dots, y_{t-1})$.
 - Sofrem com *sparsity* e contexto curto.
- O que tem funcionado melhor nos últimos anos é usar **redes neurais** para estimar essas probabilidades.

Overview

1. Modelagem de Linguagem

2. LM com Redes Neurais

3. Decodificação

4. Avaliação

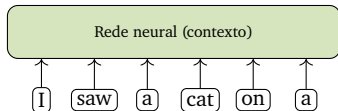
5. seq2seq

6. Atenção

7. Referências

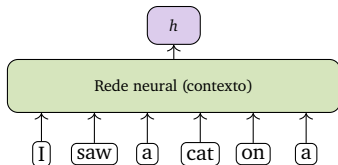
Estrutura Geral

- Modelos de linguagem com NNs costumam ter duas partes:
 - Uma rede que processa o **contexto** $y_{<t}$ produzindo um vetor h .
 - Uma camada de saída que aproxima a distribuição do **próximo token** a partir de h .
- Para processar contexto:
 - CNNs (em desuso),
 - RNNs (foco desta aula),
 - Transformers (próximas aulas).



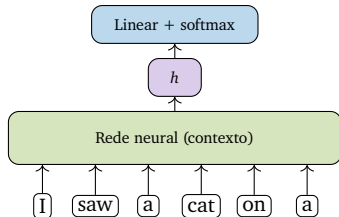
Estrutura Geral

- Modelos de linguagem com NNs costumam ter duas partes:
 - Uma rede que processa o **contexto** $y_{<t}$ produzindo um vetor h .
 - Uma camada de saída que aproxima a distribuição do **próximo token** a partir de h .
- Para processar contexto:
 - CNNs (em desuso),
 - RNNs (foco desta aula),
 - Transformers (próximas aulas).



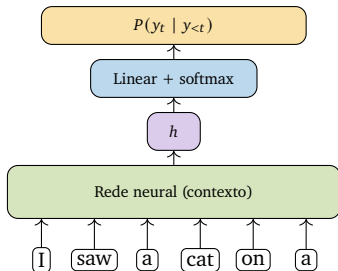
Estrutura Geral

- Modelos de linguagem com NNs costumam ter duas partes:
 - Uma rede que processa o **contexto** $y_{<t}$ produzindo um vetor h .
 - Uma camada de saída que aproxima a distribuição do **próximo token** a partir de h .
- Para processar contexto:
 - CNNs (em desuso),
 - RNNs (foco desta aula),
 - Transformers (próximas aulas).



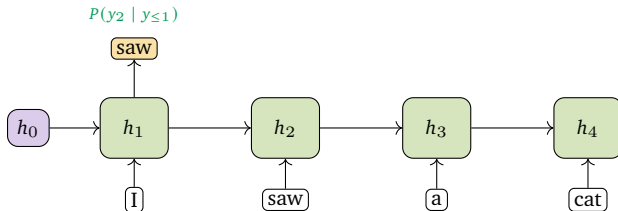
Estrutura Geral

- Modelos de linguagem com NNs costumam ter duas partes:
 - Uma rede que processa o **contexto** $y_{<t}$ produzindo um vetor h .
 - Uma camada de saída que aproxima a distribuição do **próximo token** a partir de h .
- Para processar contexto:
 - CNNs (em desuso),
 - RNNs (foco desta aula),
 - Transformers (próximas aulas).



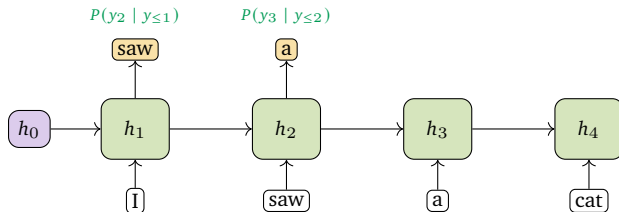
RNN como Modelo de Linguagem

- Numa RNN, o estado oculto h_t resume todo o contexto $y_{<t}$ e é atualizado a cada *token*.
- A cada passo a rede prevê o **próximo token** a partir de h_t .



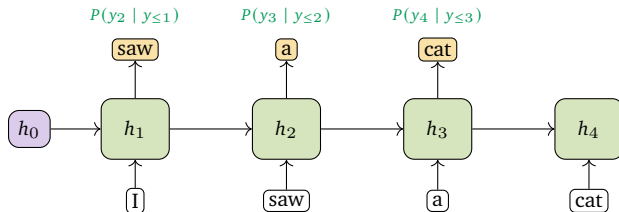
RNN como Modelo de Linguagem

- Numa RNN, o estado oculto h_t resume todo o contexto $y_{<t}$ e é atualizado a cada *token*.
- A cada passo a rede prevê o **próximo token** a partir de h_t .



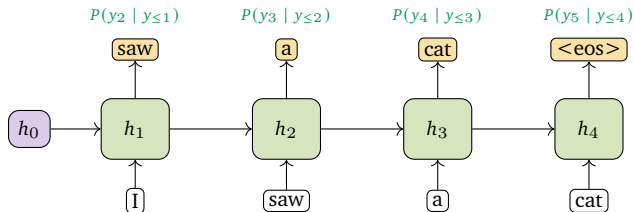
RNN como Modelo de Linguagem

- Numa RNN, o estado oculto h_t resume todo o contexto $y_{<t}$ e é atualizado a cada *token*.
- A cada passo a rede prevê o **próximo token** a partir de h_t .



RNN como Modelo de Linguagem

- Numa RNN, o estado oculto h_t resume todo o contexto $y_{<t}$ e é atualizado a cada *token*.
- A cada passo a rede prevê o **próximo token** a partir de h_t .



- O mesmo conjunto de pesos é reaplicado a cada passo (*weight sharing*), por isso a sequência pode ter qualquer comprimento.

Camada de Saída

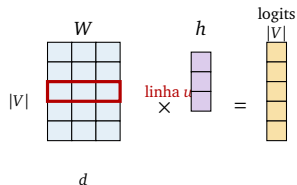
- A partir do vetor de contexto $h \in \mathbb{R}^d$ aplicamos uma transformação para a dimensão do vocabulário $|V|$.

$$\text{logits} = Wh + b, \quad W \in \mathbb{R}^{|V| \times d}$$

- Em seguida, *softmax* converte os *logits* em probabilidades:

$$P(y_t = w \mid y_{<t}) = \frac{\exp(h^\top w)}{\sum_{w_i \in V} \exp(h^\top w_i)}$$

- Cada **linha** de W é um *output word embedding*, a representação do *token* de saída.



Treinando como Classificação

- A cada passo de tempo temos um problema de classificação multiclasse com $|V|$ classes.
- Usamos *cross-entropy* como função de custo:

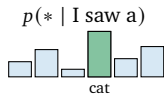
$$\mathcal{L}(y, \hat{y}) = - \sum_{i=1}^{|V|} y_i \ln(p(\hat{y}_i)) = - \ln(p(y_t \mid y_{<t}))$$

- O alvo y é um *one-hot* para o *token* correto, de modo que a soma colapsa para o termo do alvo.

Exemplo de Treinamento

Training example: I saw a **cat** on a mat <eos>

alvo: prever **cat**

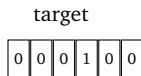


- O *backprop* ajusta os pesos para aumentar a probabilidade do *token* correto e reduzir a dos demais.

Exemplo de Treinamento

Training example: I saw a **cat** on a mat <eos>

alvo: prever **cat**

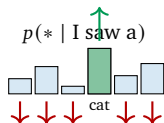


- O *backprop* ajusta os pesos para aumentar a probabilidade do *token* correto e reduzir a dos demais.

Exemplo de Treinamento

Training example: I saw a **cat** on a mat <eos>

alvo: prever **cat**



target

0	0	0	1	0	0
---	---	---	---	---	---

$$\mathcal{L} = -\log(p(\text{cat}))$$

aumenta $p(\text{cat})$

diminui os demais

- O *backprop* ajusta os pesos para aumentar a probabilidade do *token* correto e reduzir a dos demais.

Overview

1. Modelagem de Linguagem

2. LM com Redes Neurais

3. Decodificação

4. Avaliação

5. seq2seq

6. Atenção

7. Referências

Gerando Frases

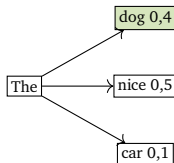
- Com um *Language Model* treinado podemos **gerar** frases.
- Esse processo também é chamado de *decoding*.
- Queremos tanto **coerência** quanto **diversidade** nas frases geradas.
 - Existe um *tradeoff* entre essas características dependendo da forma de gerar os textos.

Greedy Search

- O método mais simples: a cada passo, pegar o *token* mais provável.

$$y_t = \operatorname{argmax}_y P(y \mid y_{<t})$$

- Decisão local pode ser ruim globalmente.
- No exemplo ao lado, escolheríamos “The nice” (0,5), perdendo “The dog has” (cuja sequência tem probabilidade conjunta maior).

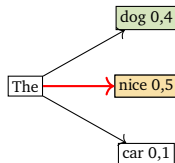


Greedy Search

- O método mais simples: a cada passo, pegar o *token* mais provável.

$$y_t = \operatorname{argmax}_y P(y \mid y_{<t})$$

- Decisão local pode ser ruim globalmente.
- No exemplo ao lado, escolheríamos “The nice” (0,5), perdendo “The dog has” (cuja sequência tem probabilidade conjunta maior).

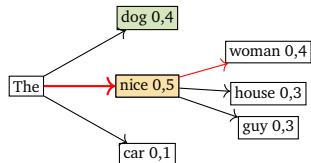


Greedy Search

- O método mais simples: a cada passo, pegar o *token* mais provável.

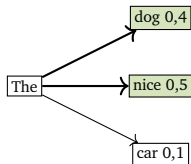
$$y_t = \operatorname{argmax}_y P(y \mid y_{<t})$$

- Decisão local pode ser ruim globalmente.
- No exemplo ao lado, escolheríamos “The nice” (0,5), perdendo “The dog has” (cuja sequência tem probabilidade conjunta maior).



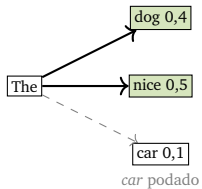
Beam Search

- Mantém os b beams mais prováveis a cada passo (*beam size* b).



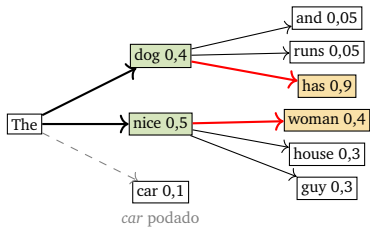
Beam Search

- Mantém os b beams mais prováveis a cada passo (*beam size* b).
- Com $b = 2$, a partir de “The” guardamos as duas continuações mais prováveis: “dog” (0,4) e “nice” (0,5).



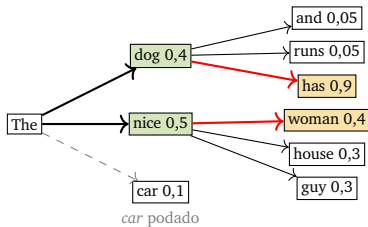
Beam Search

- Mantém os b beams mais prováveis a cada passo (*beam size* b).
- Com $b = 2$, a partir de “The” guardamos as duas continuações mais prováveis: “dog” (0,4) e “nice” (0,5).



Beam Search

- Mantém os b beams mais prováveis a cada passo (*beam size* b).
- Com $b = 2$, a partir de “The” guardamos as duas continuações mais prováveis: “dog” (0,4) e “nice” (0,5).



Candidatos $b = 2$:

The dog has $0,4 \cdot 0,9 = 0,36$

The nice woman $0,5 \cdot 0,4 = 0,20$

- Embora “The dog” fosse menos provável que “The nice”, a expansão revela “The dog has” como melhor sequência.
- Continuamos até encontrar `<eos>` ou um limite de comprimento.

Beam Search: Formulação

- O *Beam Search* busca aproximar:

$$\operatorname{argmax}_y \prod_{t=1}^n P(y_t \mid y_{<t})$$

- Por questões de **estabilidade numérica** é usual trabalhar com a soma dos *logs* das probabilidades:

$$\operatorname{argmax}_y \sum_{t=1}^n \log(P(y_t \mid y_{<t}))$$

- Como $\log$ é monotonicamente crescente, maximizar p ou $\log p$ é equivalente.

Penalidade por Comprimento

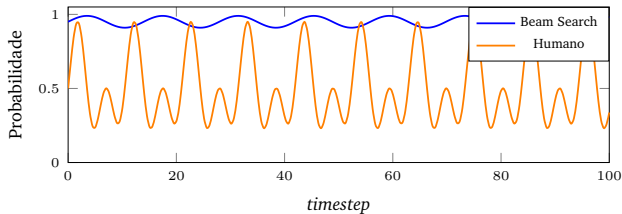
- Sequências mais longas acumulam mais $\log P$ negativos, somando *logs* a busca prefere sequências mais curtas.
- Para amenizar, dividimos pelo tamanho da sequência elevado a um hiperparâmetro $\alpha \in (0, 1)$:

$$\operatorname{argmax}_y \frac{1}{n^\alpha} \sum_{t=1}^n \log(P(y_t \mid y_{<t}))$$

- Existem outras normalizações que enviesam a busca para soluções mais longas, mais curtas, ou para maior diversidade.
- Referência prática:
https://opennmt.net/OpenNMT/translation/beam_search/.

Beam Search vs Texto Humano

- *Beam Search* funciona bem em tarefas onde o tamanho da sequência é relativamente conhecido (tradução, sumarização).
- Em geração aberta, gera saídas **repetitivas** e pouco humanas, sempre escolhe *tokens* de probabilidade alta.



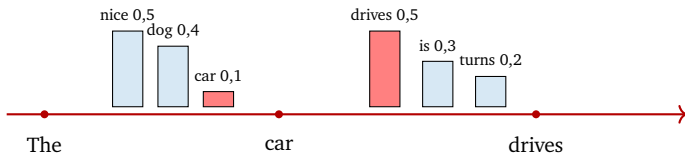
- Ilustração baseada em Holtzman et al., *The Curious Case of Neural Text Degeneration* (2019).

Sampling

- Em vez de pegar o *argmax*, podemos **amostrar** da distribuição de probabilidade:

$$y_t \sim P(y \mid y_{<t})$$

- Geração deixa de ser determinística, mas ganha em diversidade.
- Continuamos até encontrar o *token* especial `<eos>`.

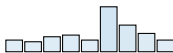


Temperatura na Softmax

- Para controlar diversidade, dividimos os *logits* por uma temperatura τ :

$$\frac{\exp(h^\top w)}{\sum_{w_i \in V} \exp(h^\top w_i)} \longrightarrow \frac{\exp\left(\frac{h^\top w}{\tau}\right)}{\sum_{w_i \in V} \exp\left(\frac{h^\top w_i}{\tau}\right)}$$

$\tau = 1$ (*standard*)



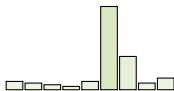
- $\tau \rightarrow 0$: tende ao *argmax* (coerência alta, diversidade baixa).
- $\tau \rightarrow \infty$: tende a uma distribuição uniforme (diversidade alta, coerência baixa).

Temperatura na Softmax

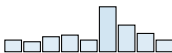
- Para controlar diversidade, dividimos os *logits* por uma temperatura τ :

$$\frac{\exp(h^\top w)}{\sum_{w_i \in V} \exp(h^\top w_i)} \longrightarrow \frac{\exp\left(\frac{h^\top w}{\tau}\right)}{\sum_{w_i \in V} \exp\left(\frac{h^\top w_i}{\tau}\right)}$$

$\tau = 0,5$ (mais nítida)



$\tau = 1$ (*standard*)



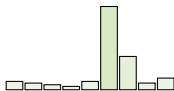
- $\tau \rightarrow 0$: tende ao *argmax* (coerência alta, diversidade baixa).
- $\tau \rightarrow \infty$: tende a uma distribuição uniforme (diversidade alta, coerência baixa).

Temperatura na Softmax

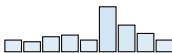
- Para controlar diversidade, dividimos os *logits* por uma temperatura τ :

$$\frac{\exp(h^\top w)}{\sum_{w_i \in V} \exp(h^\top w_i)} \longrightarrow \frac{\exp\left(\frac{h^\top w}{\tau}\right)}{\sum_{w_i \in V} \exp\left(\frac{h^\top w_i}{\tau}\right)}$$

$\tau = 0,5$ (mais nítida)



$\tau = 1$ (*standard*)



$\tau = 2$ (mais plana)

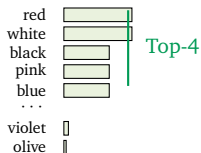


- $\tau \rightarrow 0$: tende ao *argmax* (coerência alta, diversidade baixa).
- $\tau \rightarrow \infty$: tende a uma distribuição uniforme (diversidade alta, coerência baixa).

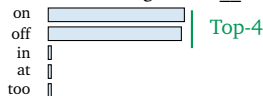
Top-K Sampling

- Heurística aplicada à amostragem:
 - Sempre amostra apenas dos K *tokens* mais prováveis.
 - Evita *tokens* pouco prováveis (que costumam ser ruídos).

Contexto: “The dress color was __”



Contexto: “The light was __”

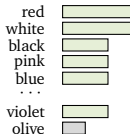


- Limitação: o K ideal depende da forma da distribuição.

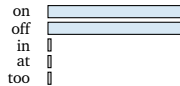
Top-P Sampling (Nucleus)

- Como fixar um K é difícil, definimos uma massa de probabilidade p .
 - Sempre amostra apenas dos *tokens* mais prováveis cuja probabilidade acumulada chega a p .

“The dress color was __” Top-80%



“The light was __” Top-80%



- O conjunto sorteado se adapta à distribuição: largo quando ela é plana, estreito quando ela é nítida.

Qual o Melhor Decoding?

- Não há consenso.
- *Top-P* é popular para geração aberta (*chatbots*, *storytelling*).
- *Beam Search* continua bastante usado em tarefas com saída bem definida (tradução, sumarização).
- Combinar técnicas (*Beam Search* + *sampling* + temperatura) também é comum.

Overview

1. Modelagem de Linguagem

2. LM com Redes Neurais

3. Decodificação

4. Avaliação

5. seq2seq

6. Atenção

7. Referências

Perplexidade

- Métrica mais usada para avaliar *Language Models*, baseada em *log-likelihood*.
- Um bom modelo atribui alta probabilidade a textos reais não vistos e baixa a textos sem sentido.
- Função de custo (*cross-entropy*) sobre a sequência:

$$\mathcal{L}(y_{1:m}) = - \sum_{t=1}^m \ln(p(y_t \mid y_{<t}))$$

- *Log-likelihood*:

$$L(y_{1:m}) = \sum_{t=1}^m \log_2(p(y_t \mid y_{<t}))$$

- Perplexidade:

$$\text{Perplexidade}(y_{1:m}) = 2^{-\frac{1}{m}L(y_{1:m})}, \quad \text{Perplexidade} \in [1, |V|]$$

Interpretando a Perplexidade

- Bons modelos têm **log-verossimilhança alta e perplexidade baixa**.
- Intuição: a perplexidade pode ser lida como o “número médio efetivo de *tokens*” entre os quais o modelo está em dúvida em cada passo.
 - Perplexidade = 1: o modelo é determinístico e está sempre certo.
 - Perplexidade = $|V|$: o modelo é equivalente a uma distribuição uniforme sobre o vocabulário.
- Cuidado ao comparar perplexidades entre modelos com tokenizadores ou vocabulários diferentes.

Exemplo Numérico: Perplexidade

- Sentença de $m = 4$ *tokens* $y_{1:4}$. Probabilidades atribuídas pelo modelo a cada *token* dado o contexto anterior:

t	1	2	3	4
$p(y_t y_{<t})$	0,5	0,25	0,5	0,5
$\log_2 p(y_t y_{<t})$	-1	-2	-1	-1

- Soma da *log-likelihood*:

$$L = -1 + (-2) + (-1) + (-1) = -5.$$

- Expoente da perplexidade:

$$-\frac{L}{m} = -\frac{-5}{4} = 1,25.$$

- Perplexidade:

$$2^{1,25} \approx 2,378.$$

- Leitura: em média, o modelo se comporta como se estivesse em dúvida entre cerca de 2,4 *tokens* a cada passo.

Overview

1. Modelagem de Linguagem
2. LM com Redes Neurais
3. Decodificação
4. Avaliação
- 5. seq2seq**
6. Atenção
7. Referências

Language Models Condicionados

- Em tarefas *seq2seq* (*sequence-to-sequence*), queremos gerar uma sequência y a partir de uma entrada x .
- Adicionamos x como condicionante:

$$P(y_1, y_2, \dots, y_n \mid x) = \prod_{t=1}^n P(y_t \mid y_{<t}, x)$$

- Mesma formulação serve para diversas tarefas:
 - Tradução automática (x é a frase no idioma de origem).
 - *Image captioning* (x é uma imagem).
 - Sumarização (x é um documento longo).
 - Resposta a perguntas (x é a pergunta).

Tradução Automática

- Dada uma sequência em um idioma, queremos descobrir uma sequência em outro idioma.
- Em termos probabilísticos:

$$y' = \operatorname{argmax}_y p(y \mid x, \theta)$$

Três perguntas a responder:

modeling

Como é o modelo
 $p(y \mid x, \theta)$?

learning

Como achar θ ?

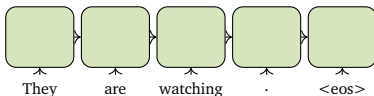
search

Como achar o argmax ?

Arquitetura Encoder-Decoder

- *Encoder-Decoder* é a arquitetura clássica de *seq2seq*.
 - O *encoder* transforma a entrada de tamanho variável em um **estado oculto**.
 - O *decoder* transforma esse estado oculto em uma sequência de saída.
 - *Tokens* especiais <bos> e <eos> marcam começo e fim.

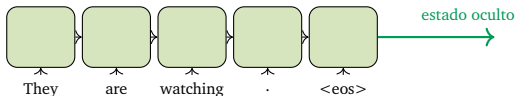
Encoder



Arquitetura Encoder-Decoder

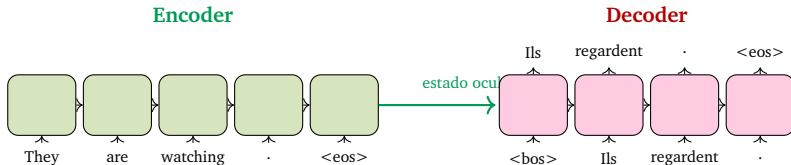
- *Encoder-Decoder* é a arquitetura clássica de *seq2seq*.
 - O *encoder* transforma a entrada de tamanho variável em um **estado oculto**.
 - O *decoder* transforma esse estado oculto em uma sequência de saída.
 - *Tokens* especiais <bos> e <eos> marcam começo e fim.

Encoder



Arquitetura Encoder-Decoder

- *Encoder-Decoder* é a arquitetura clássica de *seq2seq*.
 - O *encoder* transforma a entrada de tamanho variável em um **estado oculto**.
 - O *decoder* transforma esse estado oculto em uma sequência de saída.
 - *Tokens* especiais <bos> e <eos> marcam começo e fim.



Decoder em Treino vs Teste

- Entrar com os dados no *encoder* é trivial (a sequência de origem).
- Para o *decoder*, há mais de uma forma de alimentar a entrada:
 - **Em treino** (*teacher forcing*): condiciona no estado oculto do *encoder* e nos *tokens* precedentes do *ground truth*.
 - **Em teste**: condiciona no estado oculto do *encoder* e nos *tokens* já preditos pelo próprio modelo.
- O *decoder* é, em essência, um *Language Model* condicionado no estado oculto do *encoder*.
- Essa diferença entre treino e teste gera o chamado *exposure bias*: o modelo só vê *tokens* corretos no treino, mas no teste pode encontrar *tokens* ruins gerados por ele mesmo, e o erro se propaga.
- Uma mitigação clássica é o *scheduled sampling* (Bengio et al., 2015): durante o treino, alterna-se probabilisticamente entre o *token* do *ground truth* e o *token* predito pelo próprio modelo, suavizando a transição entre treino e inferência.

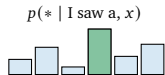
Treinando uma MT

- O treino e a amostragem em tradução automática seguem os mesmos princípios da modelagem de linguagem:
 - *Cross-Entropy Loss* a cada passo do *decoder*.
 - *Beam Search* para gerar a tradução final.

Source (FR): Je vois un chat sur un tapis <eos>

Target (EN): I saw a **cat** on a mat <eos>

previous tokens fixos no treino



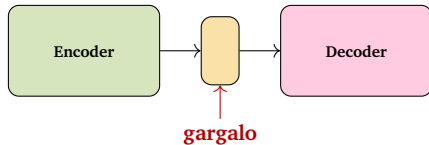
$$\mathcal{L} = -\log(p(\text{cat}))$$

- Referência seminal: *Sequence to Sequence Learning with Neural Networks*¹.

¹Ilya Sutskever, Oriol Vinyals e Quoc V. Le. “Sequence to sequence learning with neural networks”. Em: [NeurIPS](#). vol. 27. 2014.

O Problema do Gargalo

- O *encoder* comprime toda a frase de origem em **um único vetor**.
- Dois problemas:
 - É difícil para o *encoder* capturar todo o contexto da frase original.
 - É difícil para o *decoder* gerar todos os *tokens* condicionado em um contexto fixo do *encoder*.
- Esse vetor é um **gargalo** de informação.



Overview

1. Modelagem de Linguagem
2. LM com Redes Neurais
3. Decodificação
4. Avaliação
5. seq2seq
- 6. Atenção**
7. Referências

Mecanismo de Atenção

- Para resolver o gargalo entre *encoder* e *decoder* foi introduzido o mecanismo de **atenção**².
 - Um bloco que conecta *encoder* e *decoder* e atribui *scores* a cada estado do *encoder*.
 - Interpretação: a atenção decide **quais partes da entrada são mais importantes** para gerar o próximo *token*.
- Existem várias implementações, vamos ver a forma de *Bahdanau* aplicada a tradução com RNNs.

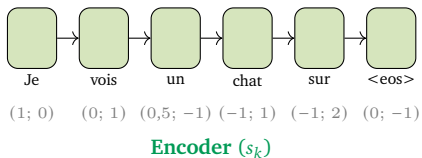
²Dzmitry Bahdanau, Kyunghyun Cho e Yoshua Bengio. "Neural machine translation by jointly learning to align and translate". Em: [arXiv preprint arXiv:1409.0473](https://arxiv.org/abs/1409.0473) (2014).

Como Funciona a Atenção

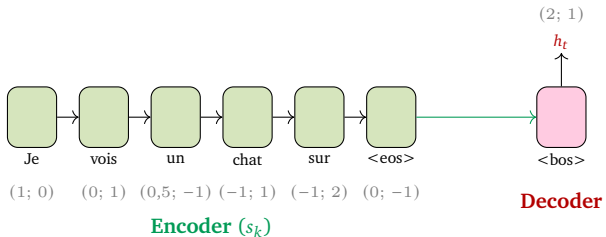
- A cada passo t do *decoder*:
 1. Recebe o estado oculto do *decoder* h_t e todos os estados do *encoder* $s_1, \dots, s_m$.
 2. Computa um *score* $\text{score}(h_t, s_k)$ para cada $k = 1, \dots, m$.
 3. Aplica *softmax* sobre os *scores* para obter os **pesos de atenção** $a_k^{(t)}$.
 4. Calcula o *context vector* $c^{(t)}$ como soma ponderada dos s_k pelos pesos.
 5. O contexto é passado para o *decoder* para a previsão do próximo *token*.

$$a_k^{(t)} = \frac{\exp(\text{score}(h_t, s_k))}{\sum_{i=1}^m \exp(\text{score}(h_t, s_i))}, \quad c^{(t)} = \sum_{k=1}^m a_k^{(t)} s_k$$

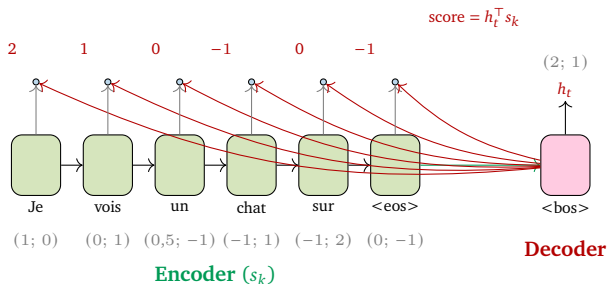
Atenção: Diagrama



Atenção: Diagrama



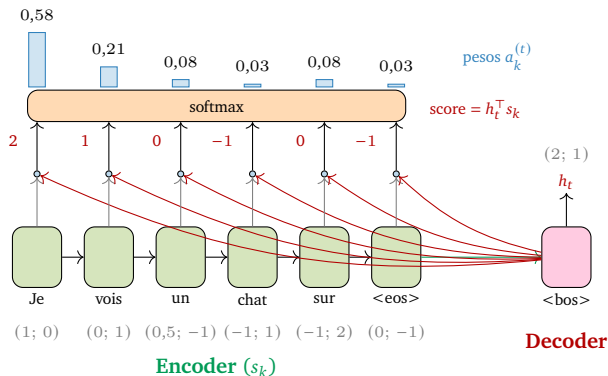
Atenção: Diagrama



Scores: $h_t^\top s_k$ com $h_t = (2; 1)$:

$$(2; 1) \cdot (1; 0) = 2, \quad (2; 1) \cdot (0; 1) = 1, \quad (2; 1) \cdot (0,5; -1) = 0 \\ (2; 1) \cdot (-1; 1) = -1, \quad (2; 1) \cdot (-1; 2) = 0, \quad (2; 1) \cdot (0; -1) = -1 \Rightarrow [2, 1, 0, -1, 0, -1]$$

Atenção: Diagrama



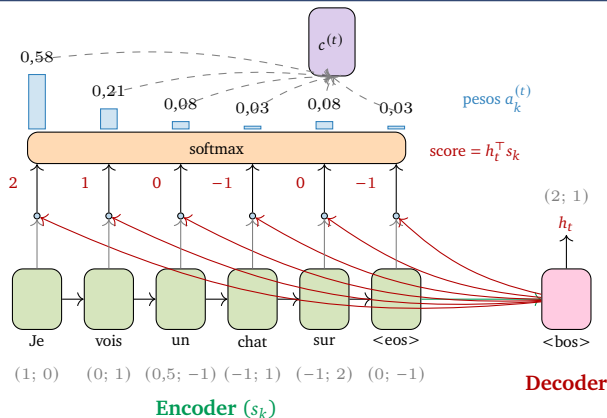
Scores: $h_t^\top s_k$ com $h_t = (2; 1)$:

$$(2; 1) \cdot (1; 0) = 2, \quad (2; 1) \cdot (0; 1) = 1, \quad (2; 1) \cdot (0,5; -1) = 0$$

$$(2; 1) \cdot (-1; 1) = -1, \quad (2; 1) \cdot (-1; 2) = 0, \quad (2; 1) \cdot (0; -1) = -1 \Rightarrow [2, 1, 0, -1, 0, -1]$$

Softmax: $\exp[\cdot] = [7,4; 2,7; 1,0; 0,4; 1,0; 0,4], \Sigma = 12,8 \Rightarrow a^{(t)} = [0,58; 0,21; 0,08; 0,03; 0,08; 0,03]$

Atenção: Diagrama



Scores: $h_t^T s_k$ com $h_t = (2; 1)$:

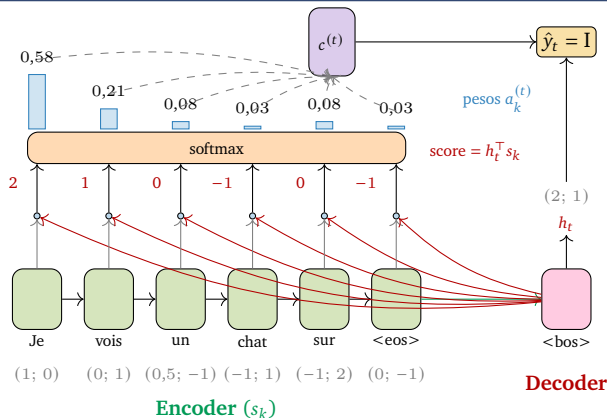
$$(2; 1) \cdot (1; 0) = 2, \quad (2; 1) \cdot (0; 1) = 1, \quad (2; 1) \cdot (0,5; -1) = 0$$

$$(2; 1) \cdot (-1; 1) = -1, \quad (2; 1) \cdot (-1; 2) = 0, \quad (2; 1) \cdot (0; -1) = -1 \Rightarrow [2, 1, 0, -1, 0, -1]$$

Softmax: $\exp[\cdot] = [7,4; 2,7; 1,0; 0,4; 1,0; 0,4], \Sigma = 12,8 \Rightarrow a^{(t)} = [0,58; 0,21; 0,08; 0,03; 0,08; 0,03]$

Contexto: $c^{(t)} = \sum_k a_k^{(t)} s_k \approx (0,51; 0,29)$

Atenção: Diagrama



Scores: $h_t^T s_k$ com $h_t = (2; 1)$:

$$(2; 1) \cdot (1; 0) = 2, \quad (2; 1) \cdot (0; 1) = 1, \quad (2; 1) \cdot (0,5; -1) = 0$$

$$(2; 1) \cdot (-1; 1) = -1, \quad (2; 1) \cdot (-1; 2) = 0, \quad (2; 1) \cdot (0; -1) = -1 \Rightarrow [2, 1, 0, -1, 0, -1]$$

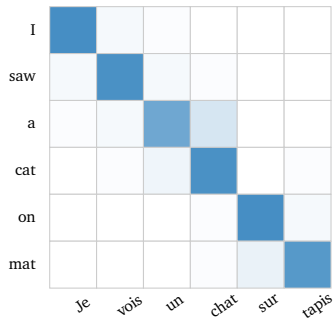
Softmax: $\exp[\cdot] = [7,4; 2,7; 1,0; 0,4; 1,0; 0,4]$, $\Sigma = 12,8 \Rightarrow a^{(t)} = [0,58; 0,21; 0,08; 0,03; 0,08; 0,03]$

Contexto: $c^{(t)} = \sum_k a_k^{(t)} s_k \approx (0,51; 0,29)$

- A cada token gerado o cálculo se repete e a distribuição de atenção se redistribui sobre a entrada (ver a matriz de alinhamento a seguir)

O Que a Atenção Aprende

- Empilhando os pesos $a_k^{(t)}$ de todos os passos formamos uma **matriz de alinhamento**.
- Cada célula mostra o quanto o *token* de saída (linha) prestou atenção ao *token* de entrada (coluna).
- O padrão quase diagonal indica que o modelo aprendeu o alinhamento entre os idiomas sem supervisão explícita.



escuro = peso de atenção alto

Variantes do Score

- Existem várias formas de calcular $\text{score}(h_t, s_k)$. As três mais clássicas:

Dot-product

$$\text{score}(h_t, s_k) = h_t^\top s_k$$

sem parâmetros

Bilinear (Luong)

$$\text{score}(h_t, s_k) = h_t^\top W s_k$$

matriz W aprendida

MLP (Bahdanau)

$$\text{score}(h_t, s_k) = w_2^\top \tanh(W_1 [h_t, s_k])$$

camada não linear

- Dot-product* é mais barato; *MLP* é mais expressivo. Em Transformers, voltaremos a versões mais sofisticadas (*scaled dot-product*)³.

³Minh-Thang Luong, Hieu Pham e Christopher D. Manning. “Effective approaches to attention-based neural machine translation”. Em: [arXiv preprint arXiv:1508.04025](https://arxiv.org/abs/1508.04025) (2015).

Overview

1. Modelagem de Linguagem
2. LM com Redes Neurais
3. Decodificação
4. Avaliação
5. seq2seq
6. Atenção
- 7. Referências**

Referências I

- Material baseado em:
 - Capítulo 10 de *Dive into Deep Learning*⁴.
 - *Excelente blog post* de Lena Voita:
https://lena-voita.github.io/nlp_course/language_modeling.html.
 - *HuggingFace blog*: <https://huggingface.co/blog/how-to-generate>.
 - Holtzman et al., *The Curious Case of Neural Text Degeneration* (2019).

- [1] Dzmitry Bahdanau, Kyunghyun Cho e Yoshua Bengio. “Neural machine translation by jointly learning to align and translate”. Em: [arXiv preprint arXiv:1409.0473](#) (2014).
- [2] Minh-Thang Luong, Hieu Pham e Christopher D. Manning. “Effective approaches to attention-based neural machine translation”. Em: [arXiv preprint arXiv:1508.04025](#) (2015).
- [3] Ilya Sutskever, Oriol Vinyals e Quoc V. Le. “Sequence to sequence learning with neural networks”. Em: [NeurIPS](#). Vol. 27. 2014.
- [4] Aston Zhang et al. [Dive into Deep Learning](#). Cambridge University Press, 2023. URL: <https://d2l.ai/>.

⁴Aston Zhang et al. [Dive into Deep Learning](#). Cambridge University Press, 2023. URL: <https://d2l.ai/>.